

Question 1 Answer: `int.Parse(null)` throws a `ArgumentNullException`.
`Convert.ToInt32(null)` returns 0.

Question 2 Answer: `TryParse` is recommended because it prevents exceptions by returning false on invalid input, making it safer and more consistent in applications.

Question 3 Answer: The purpose of `GetHashCode()` is to provide a numeric value (hash code) that helps efficiently locate objects in hash-based collections like `Dictionary` or `HashSet`.

Question 4 Answer: Reference equality in .NET checks whether two object references point to the same memory location (the exact same instance). It's significant for determining object identity, not just value similarity especially important for caching, interning, and object tracking.

Question 5 Answer: Strings are immutable in C# to ensure security, thread safety, and performance (safe sharing and interning). Internally, a string is implemented as a read-only array of characters, and any modification creates a new string instance.

Question 6 Answer: `StringBuilder` reduces inefficiencies by using a mutable buffer, avoiding the creation of multiple intermediate string objects during concatenation making it faster and more memory-efficient for repeated modifications.

Question 7 Answer: `StringBuilder` is faster for large-scale string modifications because it modifies a single buffer in memory instead of creating new string instances for each change, reducing heap allocations and garbage collection.

Question 8 Answer: String interpolation is most used because it's readable, concise, and type-safe, making code easier to write and understand.

Question 9 Answer: `StringBuilder` uses a dynamic character buffer that grows as needed, allowing in-place edits without creating new objects unlike strings, which are immutable and require a new instance for each change.